Exp. No: 1

Transformation between Cartesian, Cylindrical and Spherical Coordinate Systems

Aim :

To write a MATLAB program for transformation between Cartesian, Cylindrical and Spherical coordinate systems.

Objectives :

To understand the mathematical relationships between Cartesian, Cylindrical and Spherical coordinate systems. To develop a MATLAB program that converts the position of an object from one coordinate system to another. To verify the correctness of the coordinate transformation by comparing computed values against standard analytical formulas.

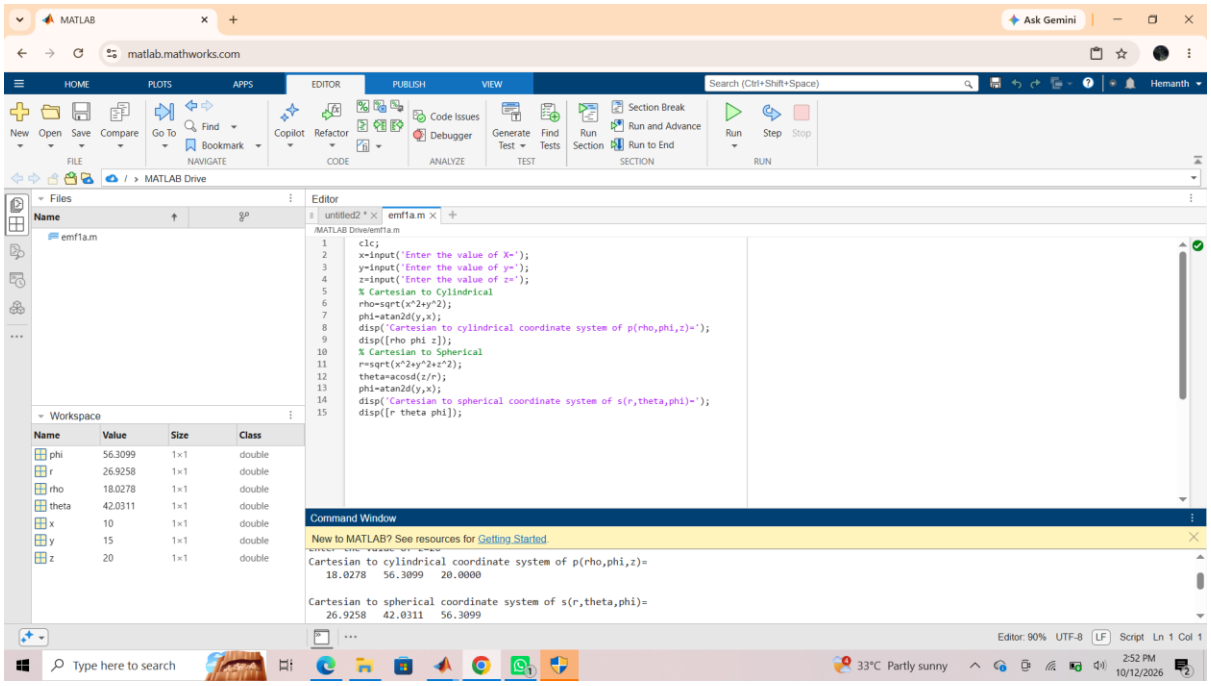
Test Case (i): Cartesian to Cylindrical and Spherical Coordinates

Input values: x=10,y=15,z=20

```
Clc;
x=input('Enter the value of X=');
y=input('Enter the value of y=');
z=input('Enter the value of z=');
% Cartesian to Cylindrical
rho=sqrt(x^2+y^2);
phi=atan2d(y,x);
disp('Cartesian to cylindrical coordinate system of p(rho,phi,z)=');
disp([rho phi z]);
% Cartesian to Spherical
r=sqrt(x^2+y^2+z^2);
theta=acosd(z/r);
phi=atan2d(y,x);
disp('Cartesian to spherical coordinate system of s(r,theta,phi)=');
```

disp([r theta phi]);

OUTPUT:



Test Case (ii): Cylindrical to Cartesian and Spherical Coordinates

Input values: ρ = 10, ϕ = 15°, z = 20

clc;

rho=input('Enter the value of rho=');

phi=input('Enter the value of phi=');

z=input('Enter the value of z=');

% Cylindrical to Cartesian

x=rho*cosd(phi);

y=rho*sind(phi);

disp('Cylindrical to Cartesian coordinate system of p(x,y,z)=');

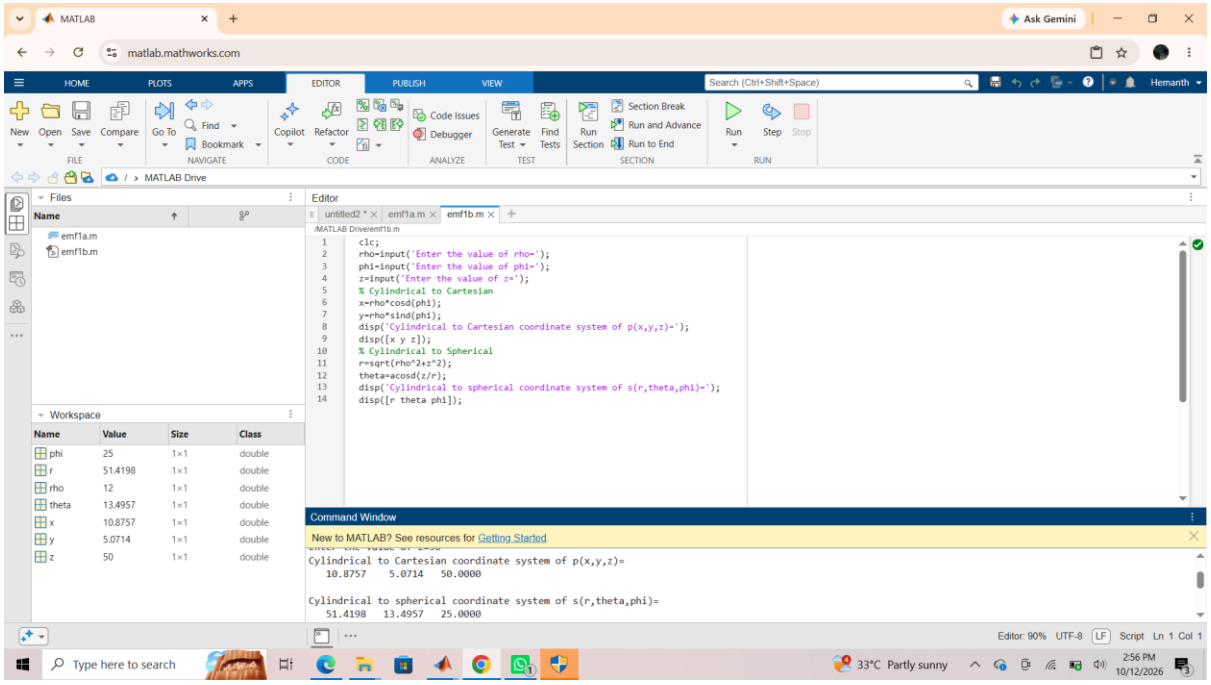
disp([x y z]);

% Cylindrical to Spherical

r=sqrt(rho^2+z^2);

```
theta=acosd(z/r);
disp('Cylindrical to spherical coordinate system of s(r,theta,phi)=');
disp([r theta phi]);
```

OUTPUT:



Test Case (iii): Spherical to Cartesian and Cylindrical Coordinates

Input values: r = 10, θ = 20°, ϕ = 15° (spherical coordinates are entered in the order r, θ, ϕ).

```
clc; r=input('Enter the value of r=');
theta=input('Enter the value of theta (deg)=');
phi=input('Enter the value of phi (deg)=');
```

% Spherical to Cartesian

x=r*sind(theta)cosd(phi);*

y=rsind(theta)sind(phi);

z=rcosd(theta);

disp('Spherical to Cartesian coordinate system of p(x,y,z)=');

disp([x y z]);

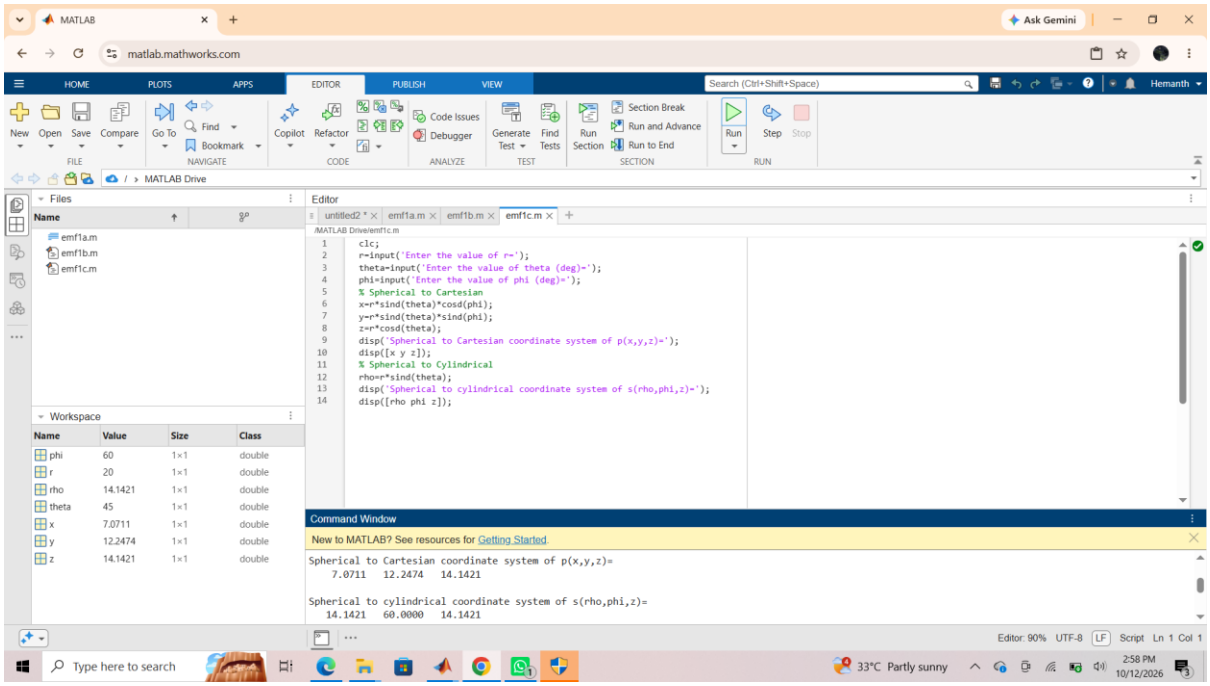
% Spherical to Cylindrical

*rho=*rsind(theta);

disp('Spherical to cylindrical coordinate system of s(rho,phi,z)=');

disp([rho phi z]);

OUTPUT:



MATLAB PROGRAM

clc; clear; close all;

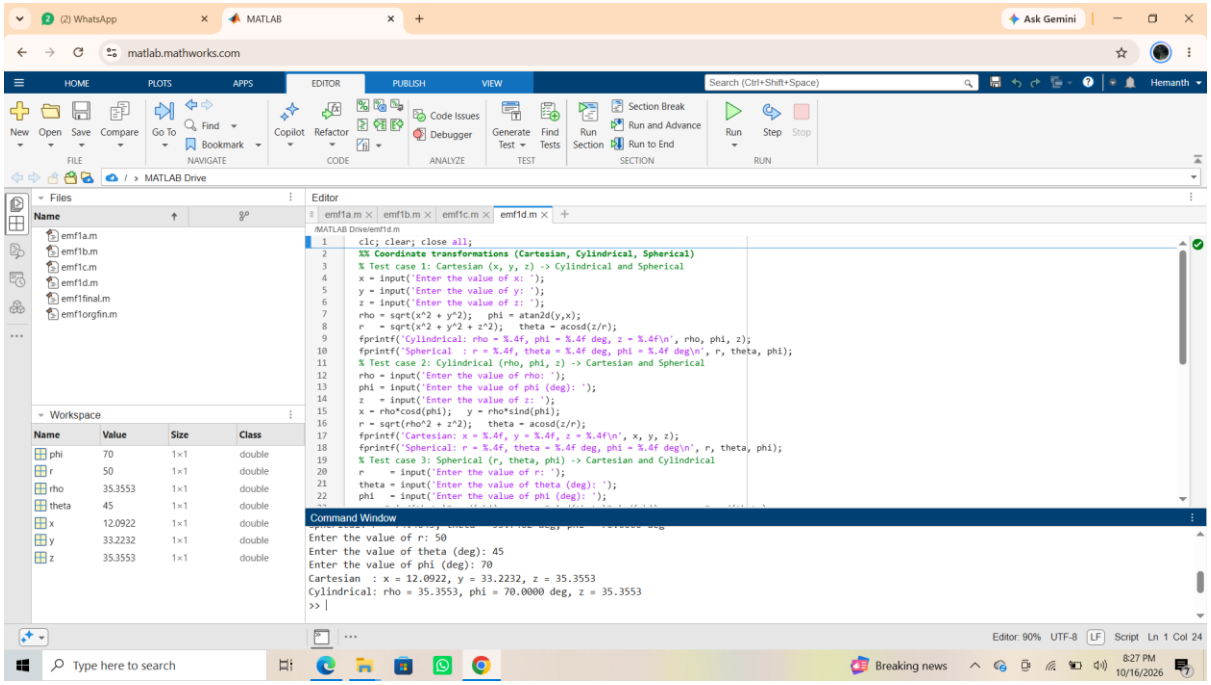
Compare MATLAB output with analytical values.

%% Coordinate transformations (Cartesian, Cylindrical, Spherical)

```
% Test case 1: Cartesian (x, y, z) -> Cylindrical and Spherical
x = input('Enter the value of x: ');
y = input('Enter the value of y: ');
z = input('Enter the value of z: ');
rho = sqrt(x^2 + y^2); phi = atan2d(y,x);
r = sqrt(x^2 + y^2 + z^2); theta = acosd(z/r);
fprintf('Cylindrical: rho = %.4f, phi = %.4f deg, z = %.4f\n', rho, phi, z);
fprintf('Spherical : r = %.4f, theta = %.4f deg, phi = %.4f deg\n', r, theta, phi);
% Test case 2: Cylindrical (rho, phi, z) -> Cartesian and Spherical
rho = input('Enter the value of rho: ');
phi = input('Enter the value of phi (deg): ');
z = input('Enter the value of z: ');
x = rho*cosd(phi); y = rho*sind(phi);
r = sqrt(rho^2 + z^2); theta = acosd(z/r);
fprintf('Cartesian: x = %.4f, y = %.4f, z = %.4f\n', x, y, z);
fprintf('Spherical: r = %.4f, theta = %.4f deg, phi = %.4f deg\n', r, theta, phi);
% Test case 3: Spherical (r, theta, phi) -> Cartesian and Cylindrical
r = input('Enter the value of r: ');
theta = input('Enter the value of theta (deg): ');
phi = input('Enter the value of phi (deg): ');
x = r*sind(theta)*cosd(phi); y = r*sind(theta)*sind(phi); z = r*cosd(theta);
rho = r*sind(theta);
fprintf('Cartesian : x = %.4f, y = %.4f, z = %.4f\n', x, y, z);
```

fprintf('Cylindrical: rho = %.4f, phi = %.4f deg, z = %.4f\n', rho, phi, z);

OUTPUT:



S. No.	Objective	Assessment Criterion	Excellent – 4	Good – 3	Satisfactory – 2	Needs Improvement – 1	Weighting (%)	Marks given	Marks obtained
1	Apply coordinate-system definitions and angle conventions	Apply coordinate-system definitions, symbols, units, and angle conventions	Applies coordinate-system definitions and angle conventions throughout.	Uses correct definitions with one minor notation or convention error.	Shows partial understanding; multiple corrections are needed.	Uses incorrect coordinate definitions or angle conventions.	25	2.5	
2	Implement transformations in MATLAB	Implement six transformations in MATLAB	Implements all six transformations correctly using suitable degree-based functions.	Implements most transformations correctly with minor coding errors.	Implements some transformations but needs assistance to correct errors.	Unable to implement the required transformations.	25	2.5	
3	Verify numerical values and reverse	Verify numerical values and reverse	All outputs agree with analytical values;	Most values agree; small unexplained	Several values differ or reverse	Outputs are incorrect or not verified.	30	3	

	transformations	transformations	reverse transformations recover the original point within rounding precision.	differences occur.	verification is incomplete.				
4	Interpret and document results	Interpret and document results	Presents complete tables, units, deviations, and a clear conclusion.	Documentation is clear with minor omissions.	Tables or interpretation are incomplete.	Results and conclusion are missing or unclear.	20	2	
Total							100	10	